



Атрибуты качества и представления архитектуры

Microservice Architecture



Проверить, идет ли запись

Меня хорошо видно & слышно?



Атрибуты качества и представления архитектуры



Чалкова Наталья Валерьевна

Архитектор, TeamLead

Об опыте:

Более 13 лет опыта Java-разработчиком и TeamLead,
Более 4 лет опыта по проектированию архитектуры.

Архитектор ФКУ «ГосТех», <https://platform.gov.ru/>

ТГ / эл. почта / соц. сети: natalia.chalkova@gmail.com
Tg: natalia_chalkova



Правила вебинара



Активно
участвуем



Off-topic обсуждаем в учебной
группе **#OTUS Microservice-2026-08**



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом

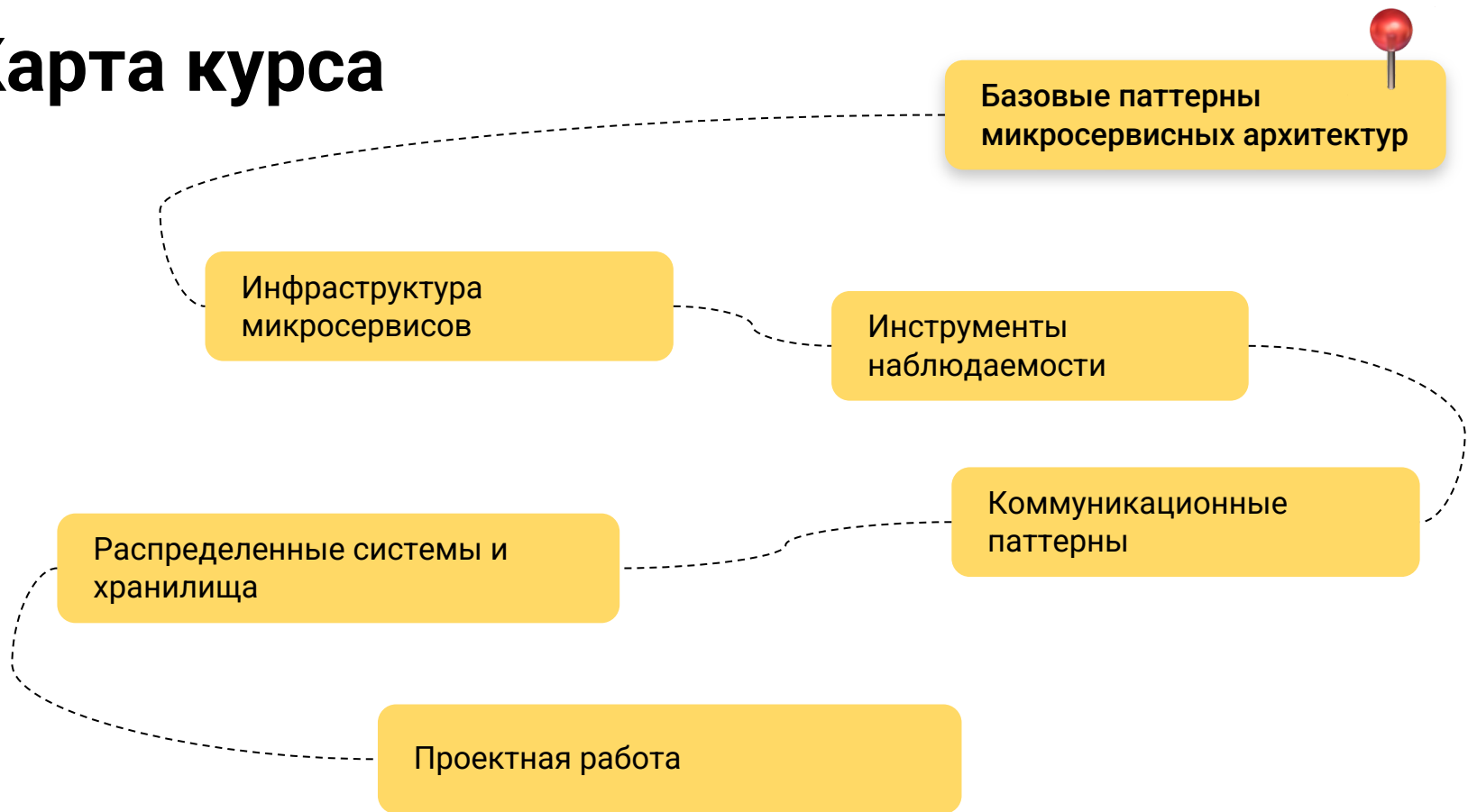


Документ



Ответьте себе или
задайте вопрос

Карта курса



Маршрут вебинара



Атрибуты качества архитектуры

Представления архитектуры

Документирование архитектурных
решений. ADR/ADL

C4 model

Итоги занятия

Цели вебинара

К концу занятия вы:

1. Поймете, что такое атрибуты качества архитектуры и как их формулировать
2. Узнаете как документировать архитектурные решения ADR/ADL в работе
3. Узнаете представления архитектуры и сможете их применять

Атрибуты качества архитектуры

Что такое стандарт, атрибут и качество?

- **Стандарт** — это правила и требования, которые помогают создавать продукт, соответствующий своему назначению
- **Атрибут** — это свойство продукта, по которому можно описать и измерить его характеристики
- **Качество** — это совокупность характеристик продукта, которые соответствуют требованиям и ожиданиям пользователей

Высокое качество достигается через следование стандартам, проверку требований и **постоянное тестирование!**

Атрибуты качества архитектуры

Атрибуты качества – это качества, которые относятся в целом ко всей системе – производительность, надежность, доступность и т.д.

Стандарты атрибутов качества ПО

- ISO/IEC 25010 (ГОСТ Р ИСО/МЭК 25010–2015)
- ISO/IEC/IEEE 29119 (IEEE 829)
- CMMI (Capability Maturity Model Integration)
- Six Sigma
- Agile Manifesto



Стандарты атрибутов качества ПО

ГОСТ Р ИСО/МЭК 25010–2015 «Информационные технологии. Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения (SQuaRE). Модели качества систем и программных продуктов» классифицирует качество программного обеспечения в виде структурированного набора характеристик.

ISO/IEC 25010

- Portability (**переносимость**)
- Functional Suitability (**функциональная пригодность**)
- Reliability (**надежность**)
- Usability (**удобство использования**)
- Security (**безопасность, защищенность**)
- Performance Efficiency (**производительность**)
- Compatibility (**совместимость**)
- Maintainability (**поддерживаемость**)



Переносимость

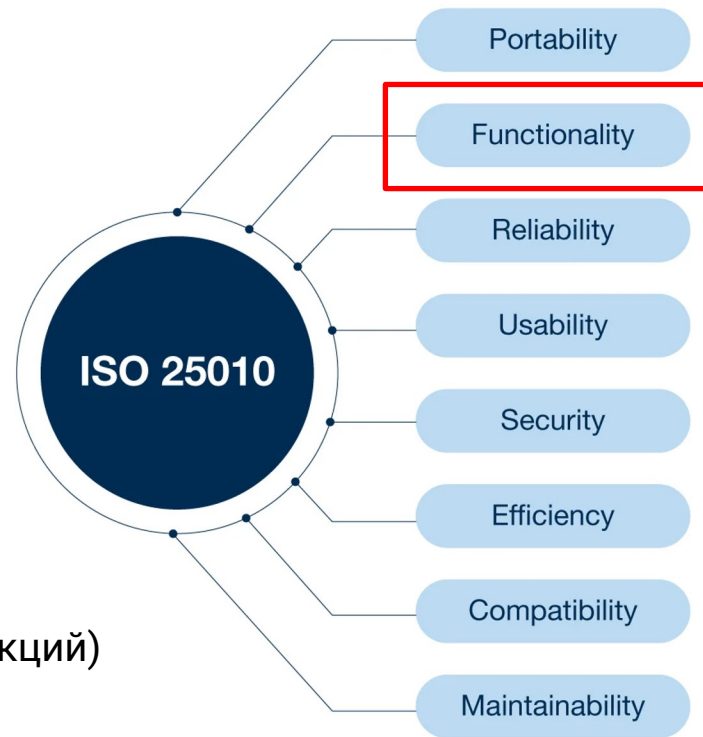


- **Adaptability** (адаптируемость)
- **Installability** (устанавливаемость)
- **Replaceability** (заменяемость)
- **Configurability** (конфигурируемость)
- **Compliance** (соответствие)
- **Documentation** (документация)
- **Performance in Different Environments** (производительность в разных окружениях)



Функциональная пригодность

- **Functional Completeness** (функц. полнота)
- **Functional Correctness** (Функц. корректность)
- **Functional Appropriateness** (Функц. уместность)
- **Error Handling** (обработка ошибок)
- **Usability of Functions** (удобство исп. функций)
- **Interoperability** (совместимость)
- **Security Functionality** (функц. безопасности)
- **Performance of Functions** (производительность функций)



Надежность

- **Maturity** (зрелость, завершенность)
- **Availability** (доступность, готовность)
- **Fault Tolerance** (устойчивость к ошибкам)
- **Recoverability** (восстанавливаемость)
- **Error Frequency** (частота ошибок)
- **Mean Time Between Failures** (MTBF, среднее время между сбоями)
- **Mean Time To Repair** (MTTR, среднее время на восстановление)
- **Consistency** (согласованность)



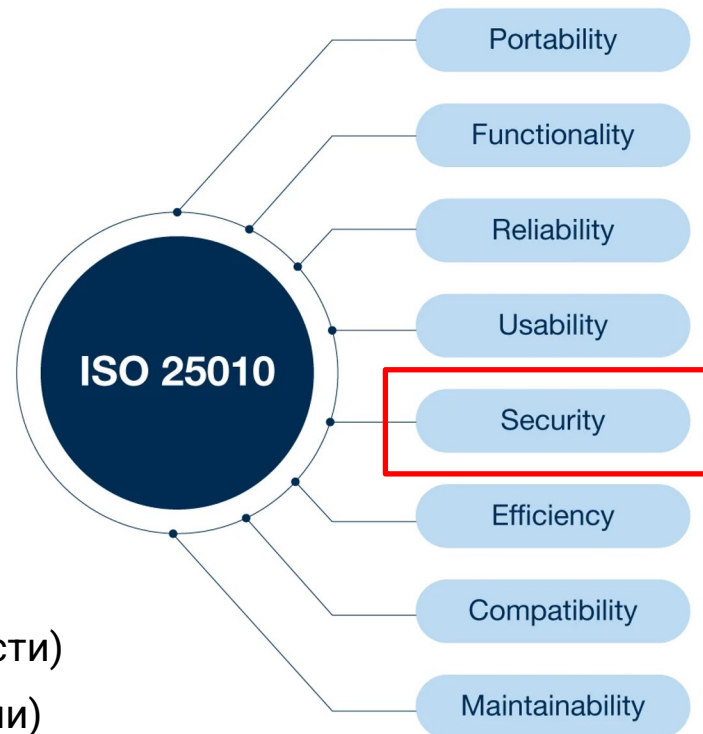
Удобство использования

- **Learnability** (обучаемость)
- **Efficiency** (эффективность)
- **Memorability** (запоминаемость)
- **Error Rate** (частота ошибок)
- **Satisfaction** (удовлетворенность)
- **Accessibility** (доступность)
- **Consistency** (согласованность)
- **User Control** (контроль пользователя)



Безопасность/защищенность

- **Confidentiality** (конфиденциальность)
- **Integrity** (целостность)
- **Availability** (доступность)
- **Authentication** (аутентификация)
- **Authorization** (авторизация)
- **Non-repudiation** (недоказуемость)
- **Auditability** (аудитируемость)
- **Security Awareness** (осведомленность о безопасности)
- **Vulnerability Management** (управление уязвимостями)



Производительность

- **Time Behavior** (временное поведение)
- **Resource Utilization** (использование ресурсов)
- **Throughput** (пропускная способность)
- **Scalability** (масштабируемость)
- **Efficiency Under Load** (эффективность под нагрузкой)
- **Latency** (задержка)
- **User Perception** (восприятие пользователем)



СОВМЕСТИМОСТЬ

- **Interoperability** (взаимодействие)
- **Compliance with Standards** (соответствие стандартам)
- **Backward Compatibility** (обратная совместимость)
- **Forward Compatibility** (перспективная совм.)
- **Platform Compatibility** (совм. с платформами)
- **Configuration Compatibility** (совм. конфигураций)
- **Data Compatibility** (совместимость данных)
- **User Environment Compatibility** (совместимость с пользовательской средой)



Поддерживаемость

- **Modularity** (модульность)
- **Reusability** (повторное использование)
- **Analyzability** (анализируемость)
- **Modifiability** (модифицируемость)
- **Testability** (тестируемость)
- **Documentation** (документация)
- **Fault Tolerance** (устойчивость к ошибкам)
- **Maintenance Effort** (усилия на поддержку)
- **Configuration Management** (управление конфигурациями)



Атрибуты качества архитектуры

Универсальных атрибутов качества архитектуры не бывает

affordability	agility	auditability	autonomy	availability
compatibility	composability	configurability	correctness	credibility
customizability	debugability	degradability	determinability	demonstrability
dependability	deployability	discoverability	distributability	durability
effectiveness	efficiency	usability	extensibility	failure transparency
fault tolerance	fidelity	flexibility	inspectability	installability
integrity	interoperability	learnability	maintainability	manageability
mobility	modifiability	modularity	operability	orthogonality
portability	precision	predictability	process capabilities	producibility
provability	recoverability	relevance	reliability	repeatability
reproducibility	resilience	responsiveness	reusability	robustness
safety	scalability	seamlessness	self-sustainability	serviceability
securability	simplicity	stability	standards compliance	survivability
sustainability	tailorability	testability	timeliness	traceability

Auditability



Performance



Security



Requirements

Data



Legality



Scalability

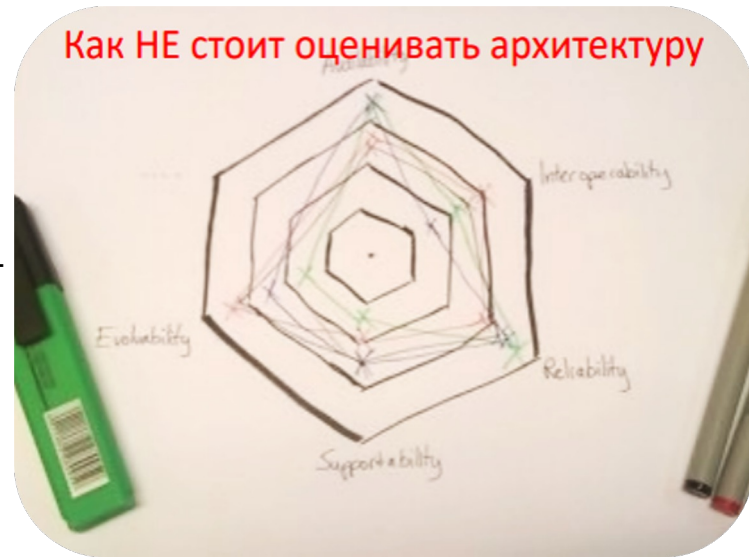


Атрибуты качества

Атрибуты качества не являются абстрактными. Не бывает абстрактной доступности, безопасности, производительности.

За каждым атрибутом качества стоит конкретный сценарий и конкретная характеристика. И любой атрибут качества проявляется в изменении характеристик или поведении информационной системы.

И именно с этими конкретными характеристиками и сценариями нужно работать.



Формат сценариев для атрибутов качества

Сценарий атрибута качества:

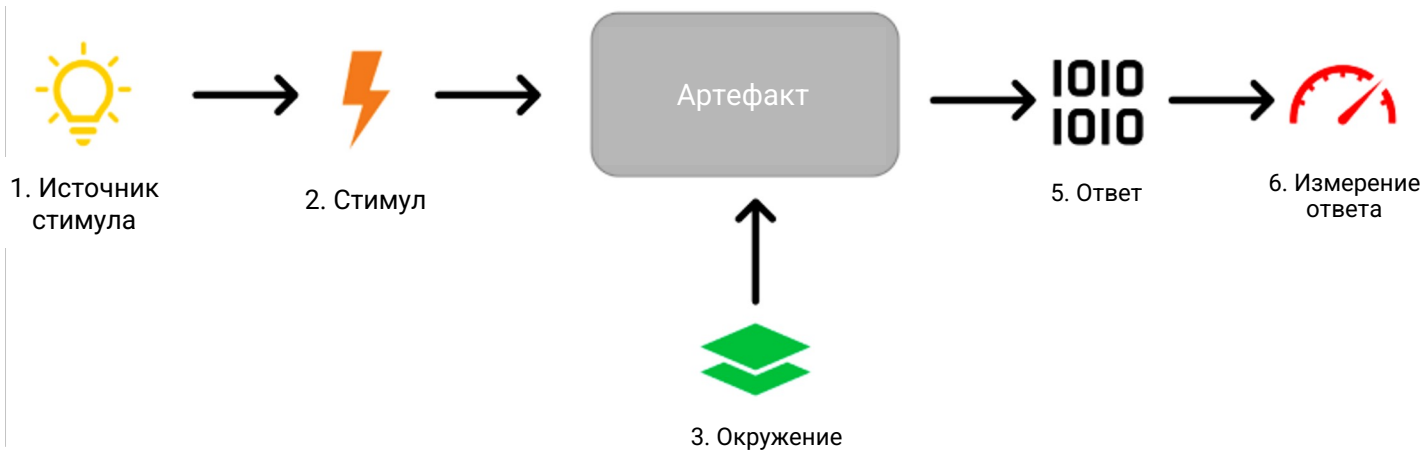
- **источник воздействия** – субъект (человек, компьютерная система или объект, обладающий поведением), генерирующий воздействие
- **воздействие** – внутренний или внешний фактор, вызывающий реакцию системы
- **окружение** – условия , в которых происходит наблюдаемое воздействие. Система может быть в рабочем режиме, работать с перегрузкой.
- **артефакт** – то, на что оказывается воздействие. Совокупность систем, вся система, часть системы
- **реакция** – ответ на пришедшее воздействие
- **мера реакции (характеристика)** – когда происходит реакция, она должна быть измерена, чтобы сравнить с исходными требованиями

Формат сценариев для атрибутов качества

Чаще всего формат сценариев сокращают до:

- Стимул
- Ответ
- Окружение

Формат сценариев для атрибутов качества



Дерево полезности (utility tree)

Дерево полезности (utility tree)

В качестве одного из инструментов для работы с атрибутами качества служит **дерево полезности (utility tree)**.

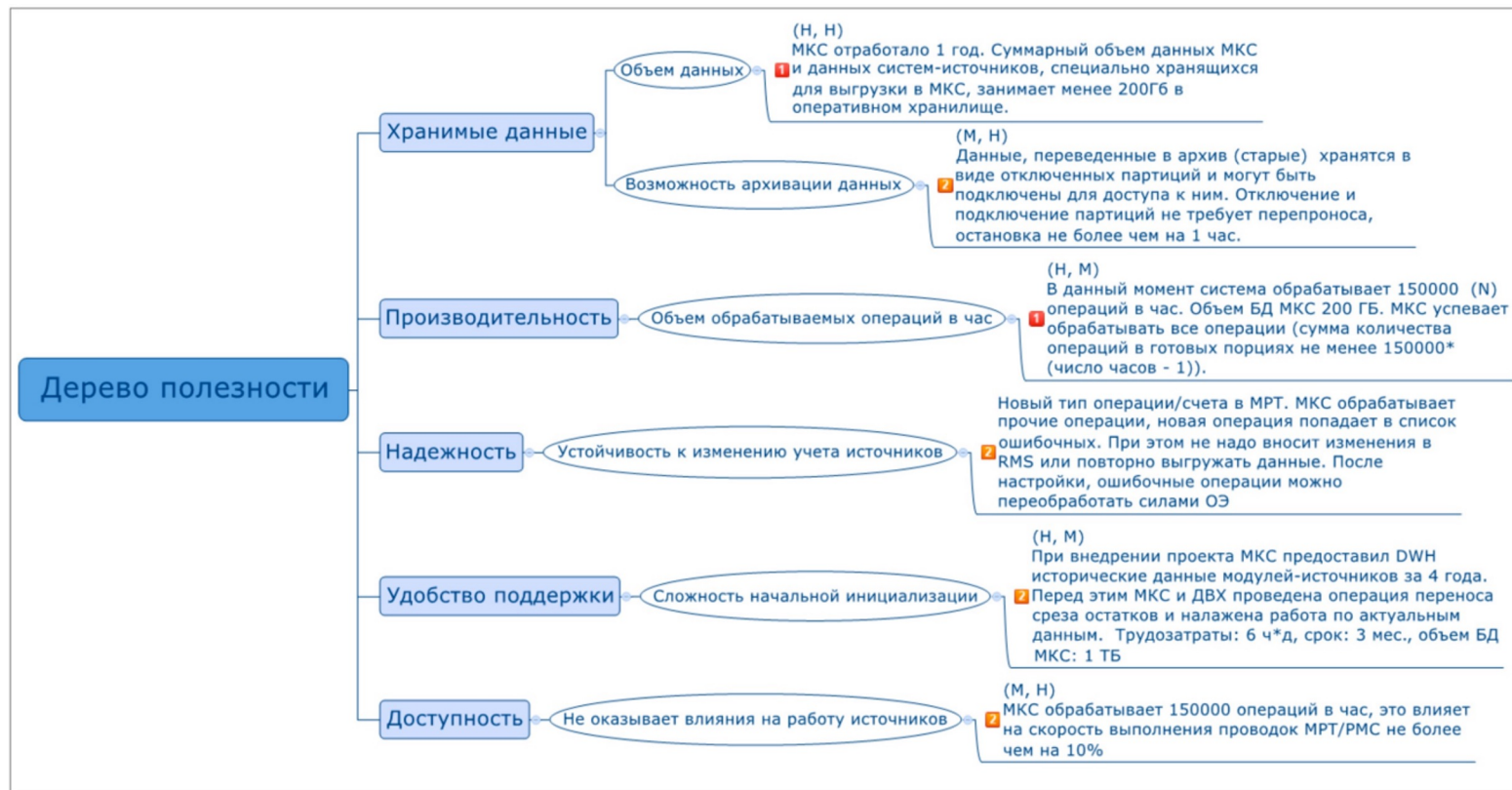
На первом уровне выбираются самые важные атрибуты качества. Например,

- Производительность
- Модифицируемость
- Безопасность
- Доступность

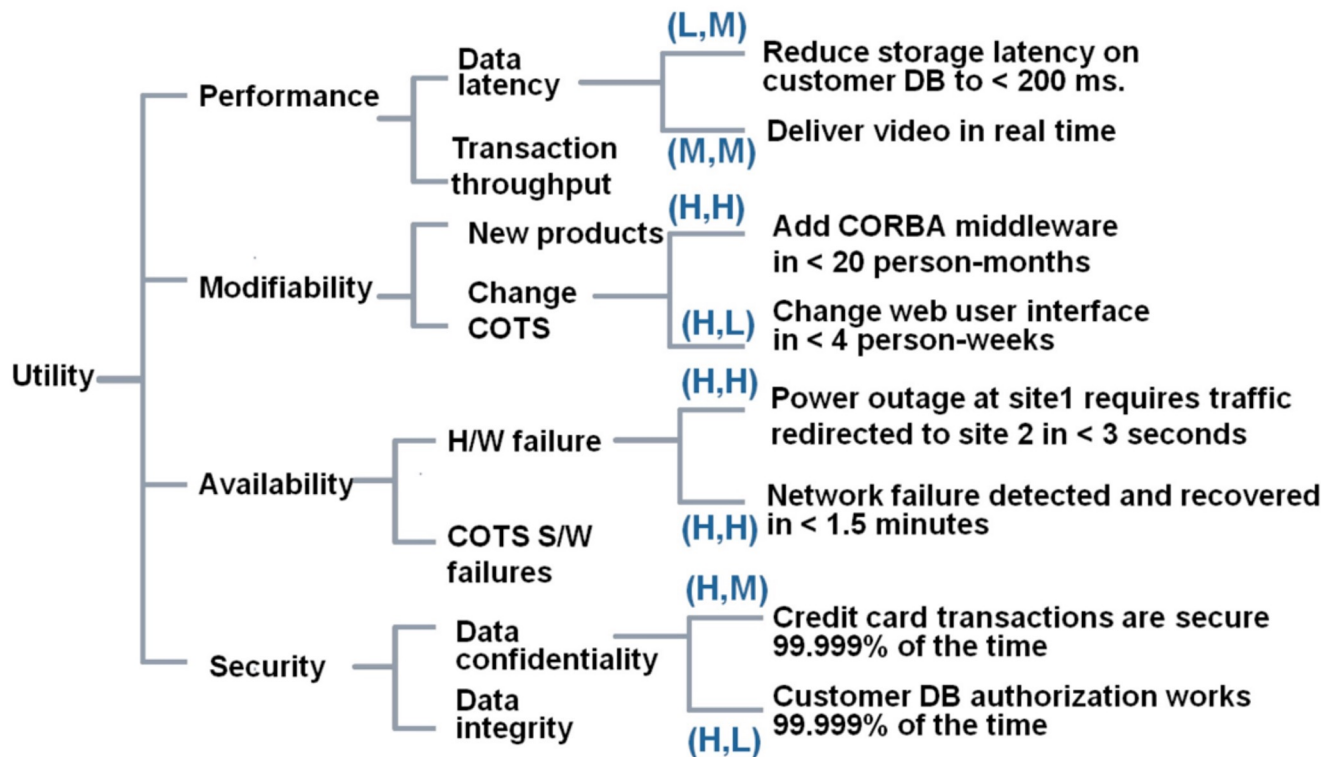
На втором уровне, для каждого атрибута выбирается ряд важных характеристик

На третьем уровне, для каждой характеристики составляется ряд сценариев

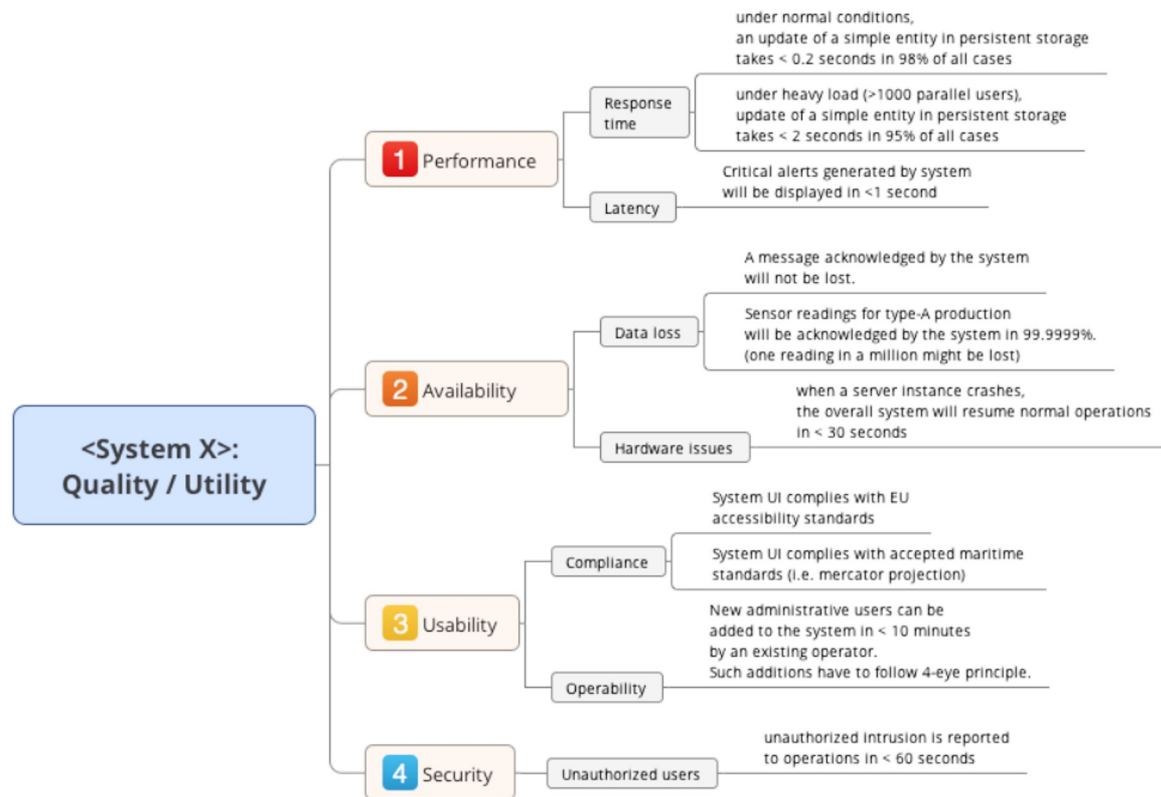
Дерево полезности



Дерево полезности



Дерево полезности



Дерево полезности (utility tree)

Есть несколько видов сценариев

- Сценарии использования
- Сценарии роста
Через 5 лет база вырастет до 1 Тб
- Сценарии исследования
Если упало половина серверов, то ...

Вопросы?

 если есть вопросы

 если вопросов нет

Представления архитектуры

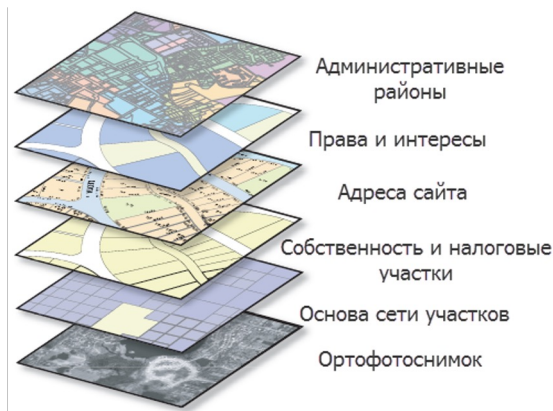
**Фактов не существует,
есть только
интерпретации фактов**

Фридрих Ницше, философ

Представления описания архитектуры

Когда архитектор описывает систему и решение, он описывает ее не максимально подробно, а показывает некоторую «проекцию», взгляд с одной из сторон.

Самая большая проблема при описании архитектур **в смешивании разных взглядов в одном описании.**





Вспомним прошлое занятие.
Для кого нужна архитектура
решения?

Архитектурные ракурсы в организации

To address the concerns of the following stakeholders...					
Users, Planners, Business Management	Database Designers and Administrators, System Engineers	System and Software Engineers	Acquirers, Operators, Administrators, & Managers		
... the following views may be developed					
Business Architecture Views	Data Architecture Views	Applications Architecture Views	Technology Architecture Views		
Business Function View	Data Entity View	Software Engineering View	Networked Computing/ Hardware View		
Business Services View					
Business Process View					
Business Information View					
Business Locations View	Data Flow View (Organization Data Use)	Applications Interoperability View	Communications Engineering View		
Business Logistics View			Processing View		
People View (Organization Chart)					
Workflow View					
Usability View				Logical Data View	Software Distribution View
Business Strategy and Goals View					
Business Objectives View					
Business Rules View					
Business Events View	Standards View				
Business Performance View					
	System Engineering View				
Enterprise Security View					
Enterprise Manageability View					
Enterprise Quality of Service View					
Enterprise Mobility View					

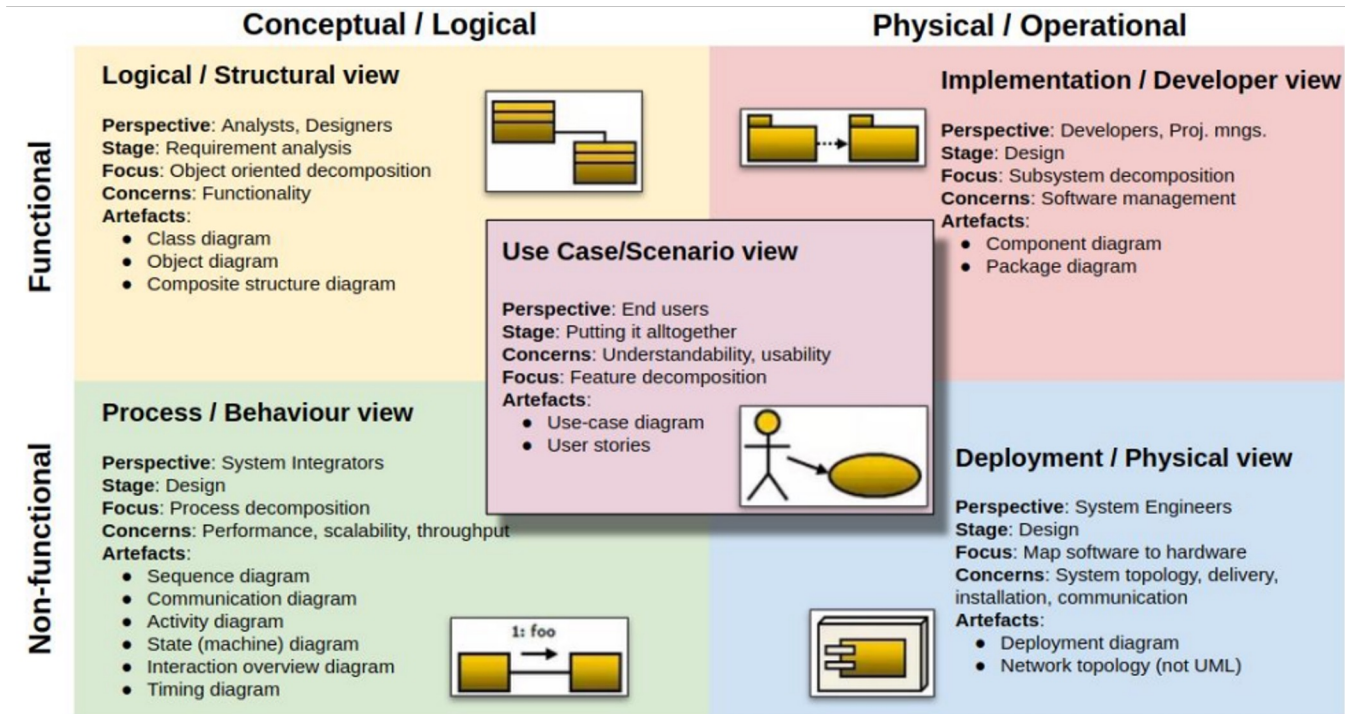
Архитектура предприятия





С какими слоями архитектуры вы работаете?

Представления описания архитектуры. 4+1

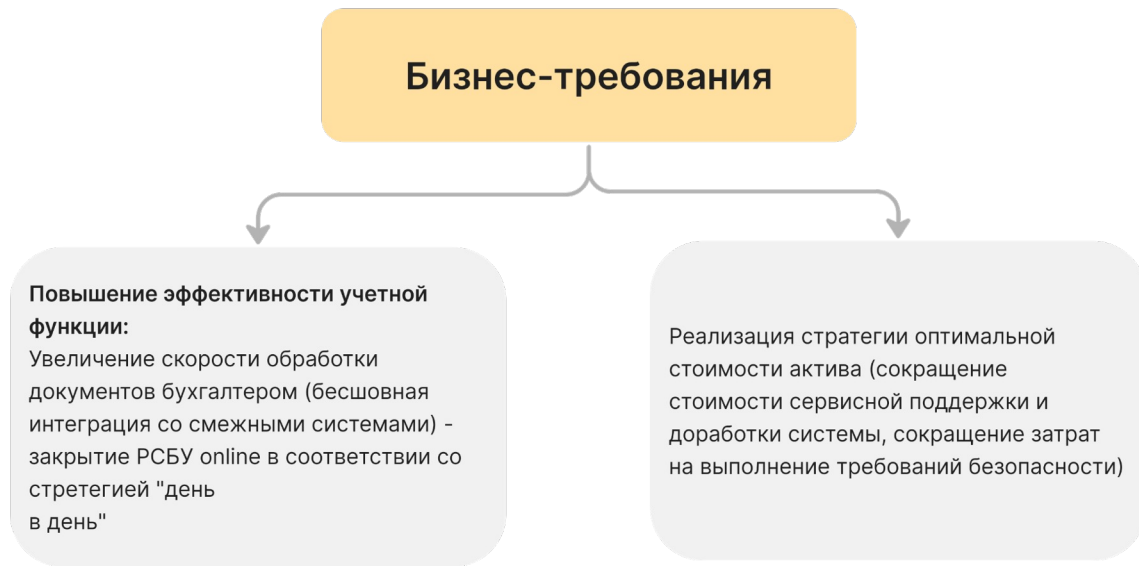


Вопросы?

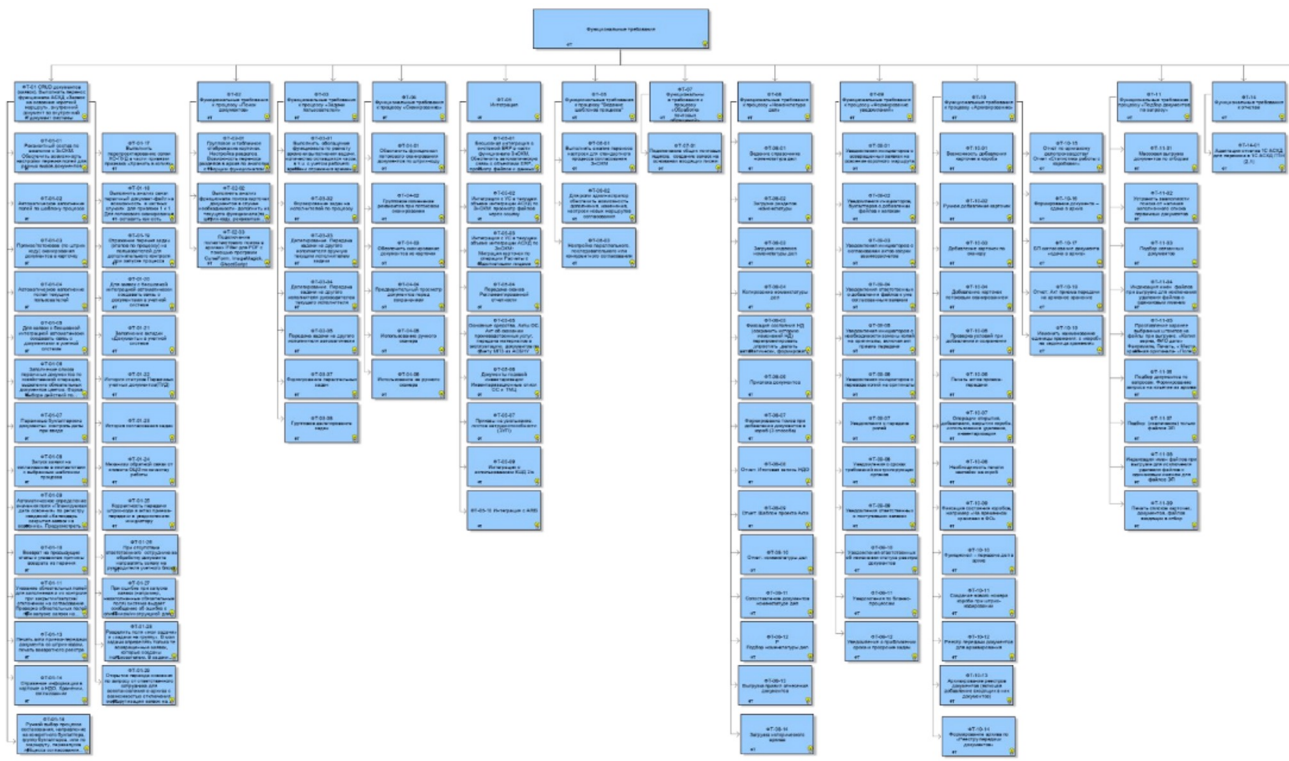
 если есть вопросы

 если вопросов нет

Формулировка бизнес-требований



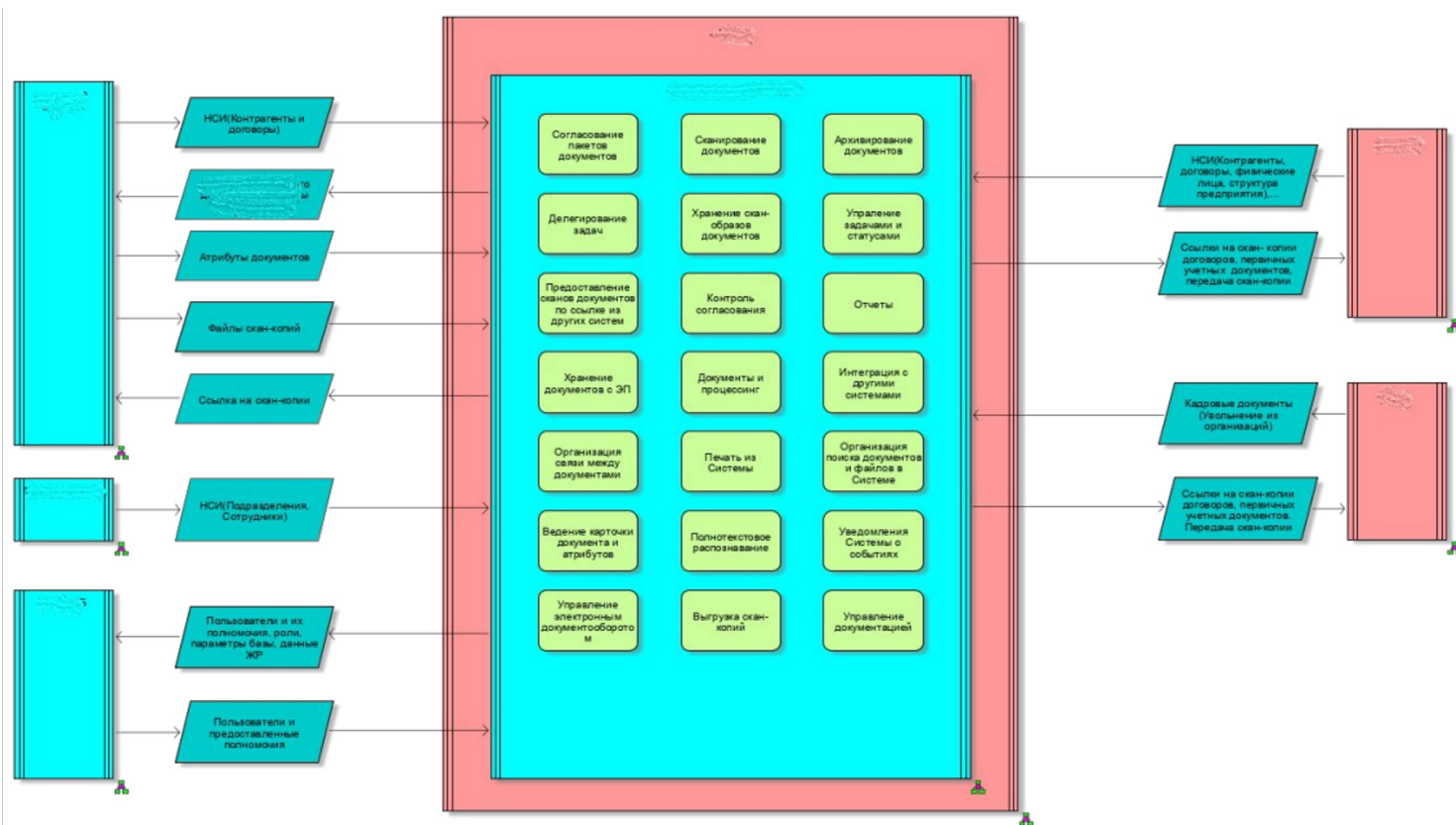
Формулировка функциональных требований



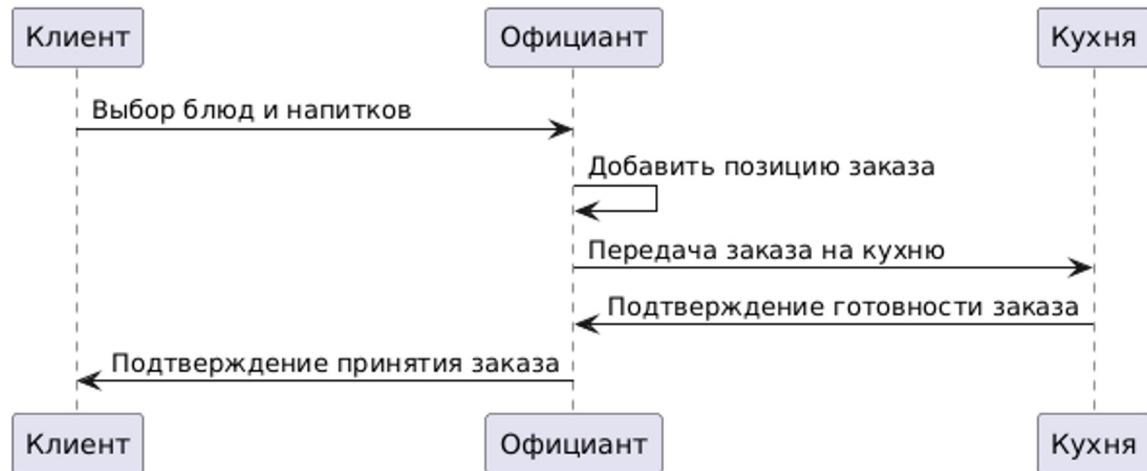
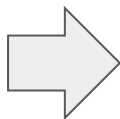
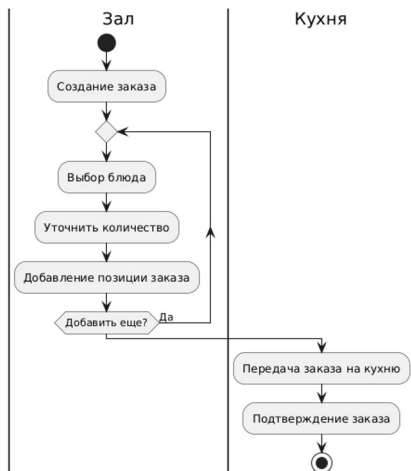
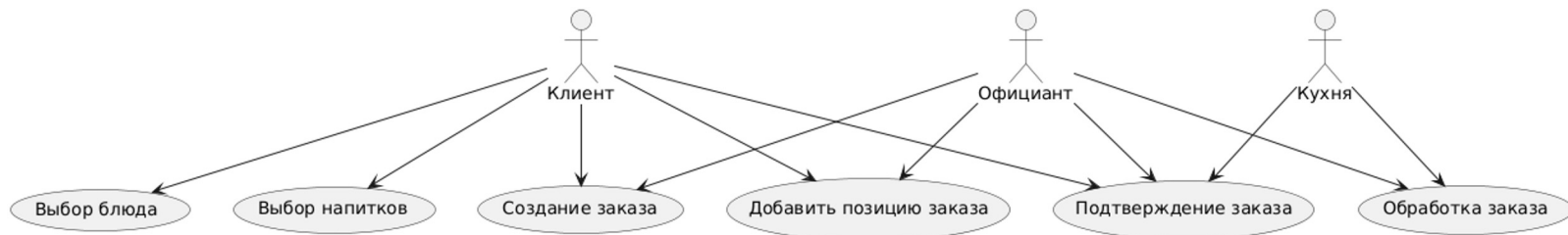
Преобразование БТ в ФТ

ФТ БТ		Повышение эффективности учетной функции: Ув...	Реализация стратегии оптимальной стоимости ак...
■ ФТ-01 CRUD документов (заявок). Выполнить перенос функционала АСХД «Заявок на осво...		✓	
■ ФТ-01-01 Реквизитный состав по аналогии с ЗНОКМ. Обеспечить возможность настройки пе...		✓	✓
■ ФТ-01-02 Автоматическое заполнение полей по шаблону процесса		✓	
■ ФТ-01-03 Прямое/потокное (по штрих-коду) сканирования документов в карточку		✓	
■ ФТ-01-04 Автоматическое заполнение полей текущим пользователями		✓	
■ ФТ-01-05 Для заявок с бесшовной интеграцией автоматически создавать связь с документа...		✓	
■ ФТ-01-06 Заполнение списка первичных документов по хозяйственной операции, выделени...		✓	
■ ФТ-01-07 Первичные бухгалтерские документы- контроль даты при вводе		✓	
■ ФТ-01-08 Запуск заявки на согласование в соответствии с выбранным шаблоном процесса		✓	
■ ФТ-01-09 Автоматическое определение значения поля «Планируемая дата освоения» по р...		✓	
■ ФТ-01-10 Возврат на предыдущие этапы с указанием причины возврата из перечня		✓	
■ ФТ-01-11 Указание обязательных полей для заполнения и их контроля при закрытии/запус...		✓	
■ ФТ-01-13 Печать акта приема-передачи документа со штрих-кодом, печать возвратного рее...		✓	
■ ФТ-01-14 Отражение информации в карточке о НДО, Хранении, согласовании		✓	

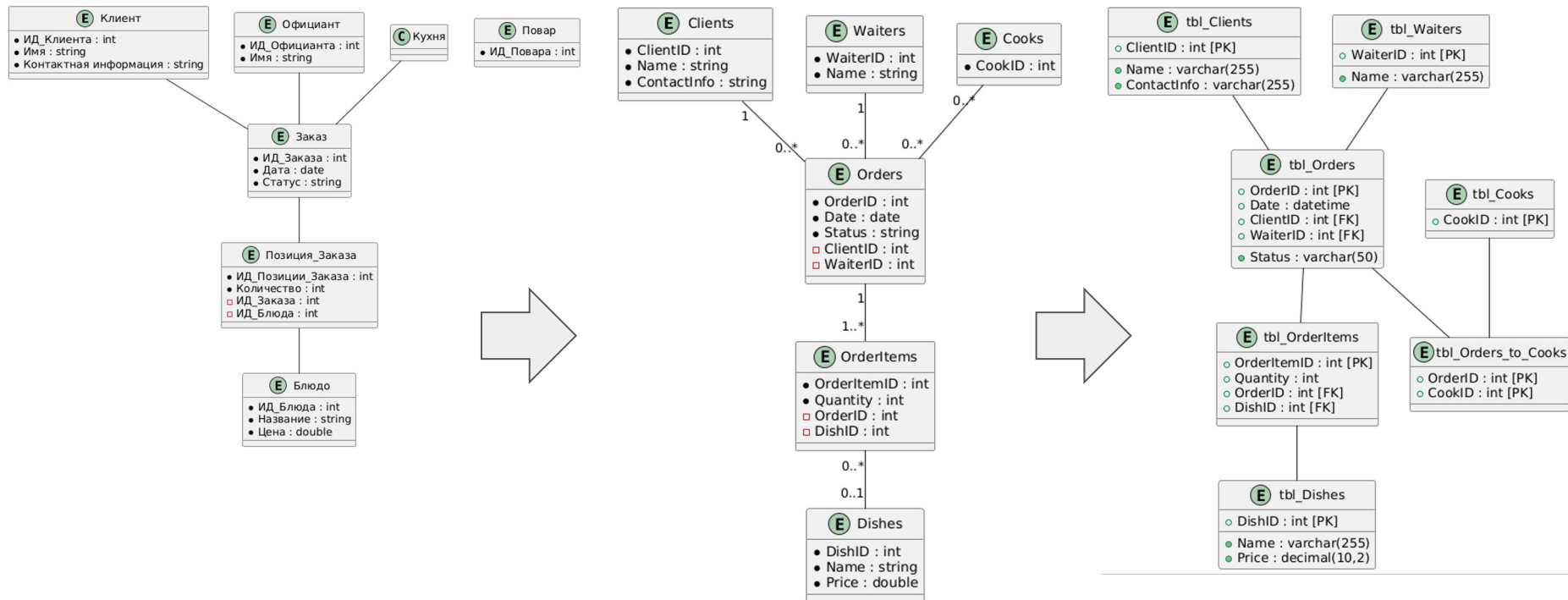
Формирование потоков данных



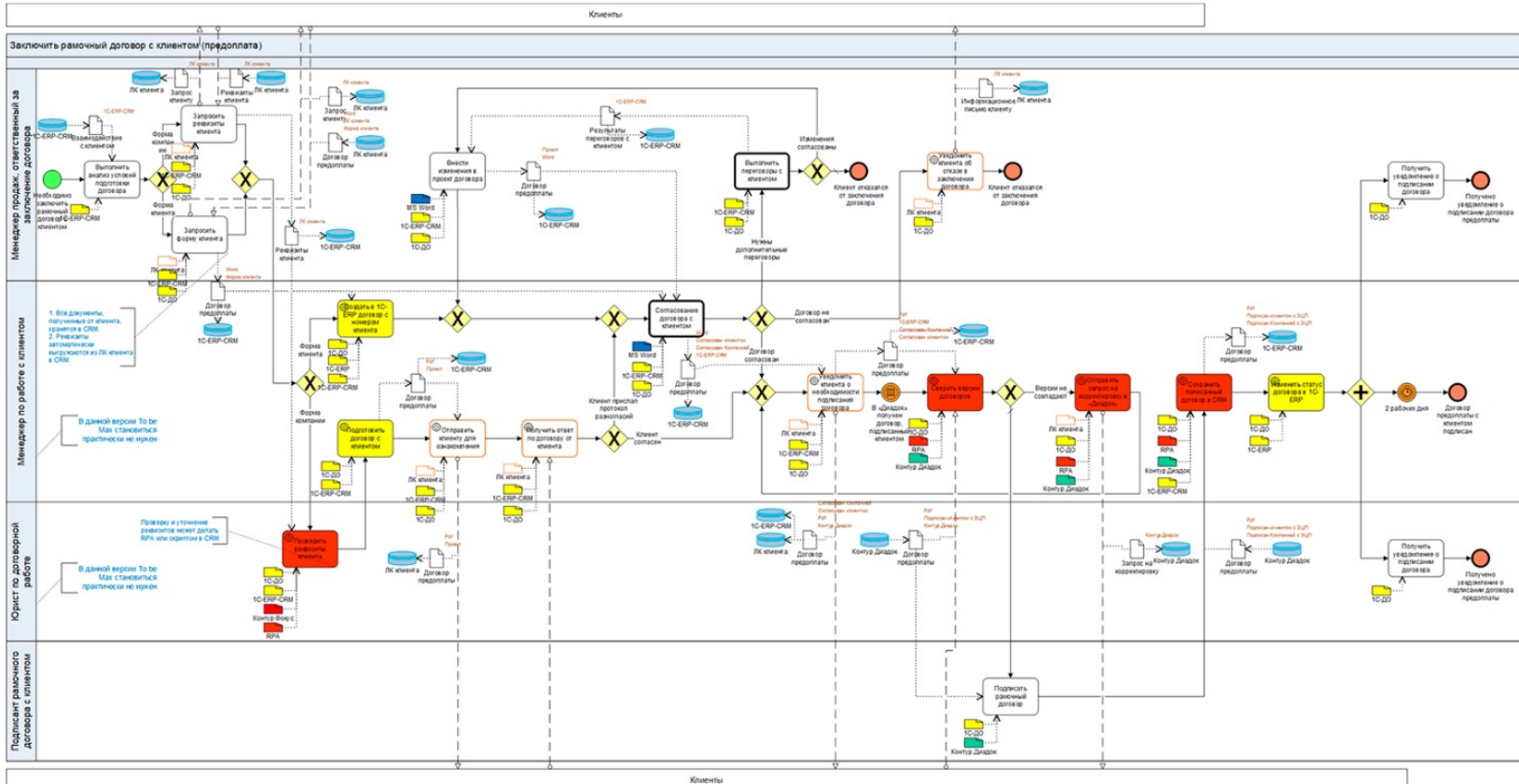
Описание сценариев работы



Создание модели данных



Описание функциональных возможностей



Ключевые UML диаграммы для архитектора

Class Diagram – структура доменной модели:

- Сущности (Entities), объекты-значения (Value Objects), сервисы
- Отношения и зависимости
- Паттерны проектирования

Sequence Diagram – детальные сценарии:

- Синхронные и асинхронные вызовы
- Поток обработки ошибок
- Временные ограничения

Ключевые UML диаграммы для архитектора

Component Diagram – модульная архитектура:

- Зависимости между модулями
- Предоставляемые/требуемые интерфейсы
- Отражает структуру

State Diagram – жизненный цикл сущности.

Отражает переход по статусам

Class Diagram: доменная модель e-commerce

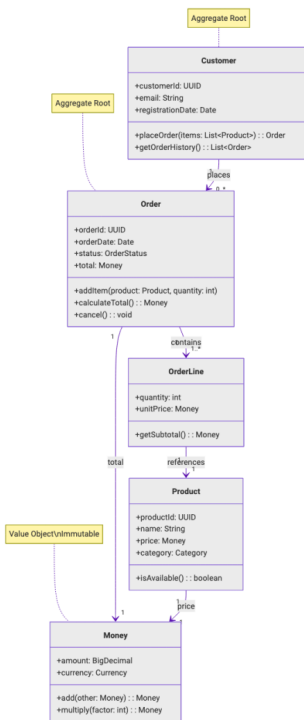
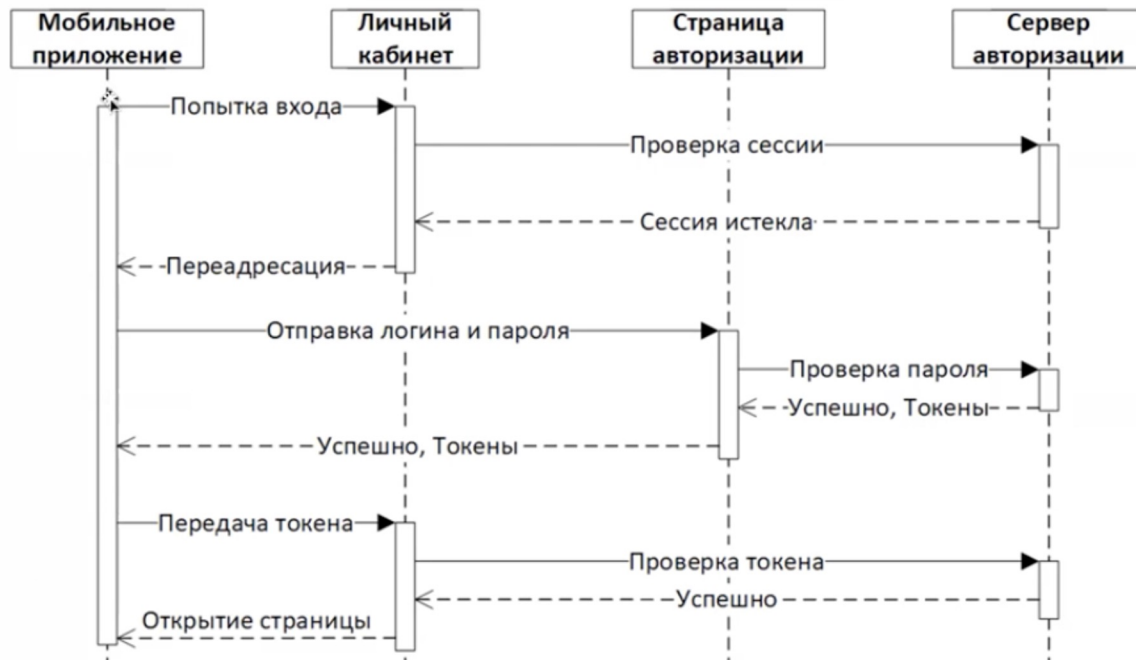
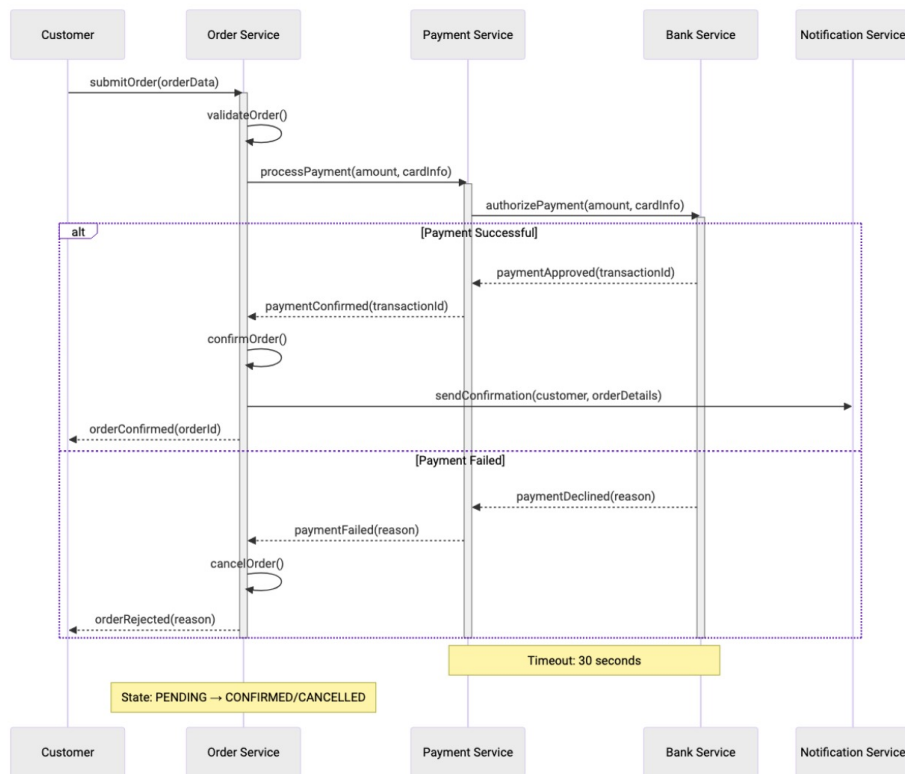


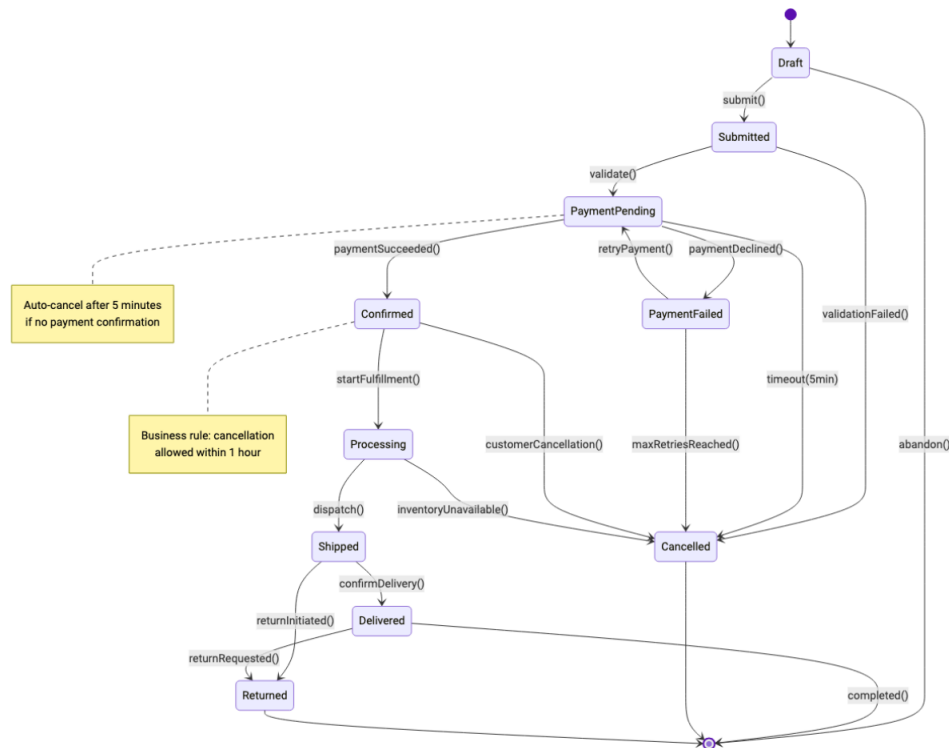
Диаграмма последовательности (sequence diagram)



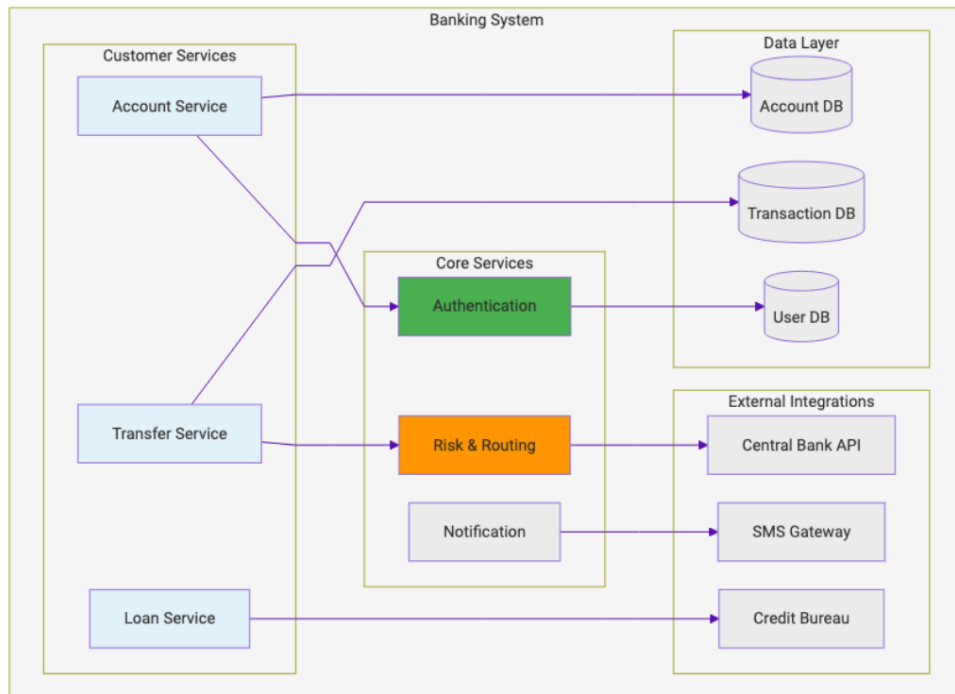
Sequence diagram: обработка платежа



State diagram: жизненный цикл заказа



UML Component: banking system architecture



Когда использовать UML диаграммы

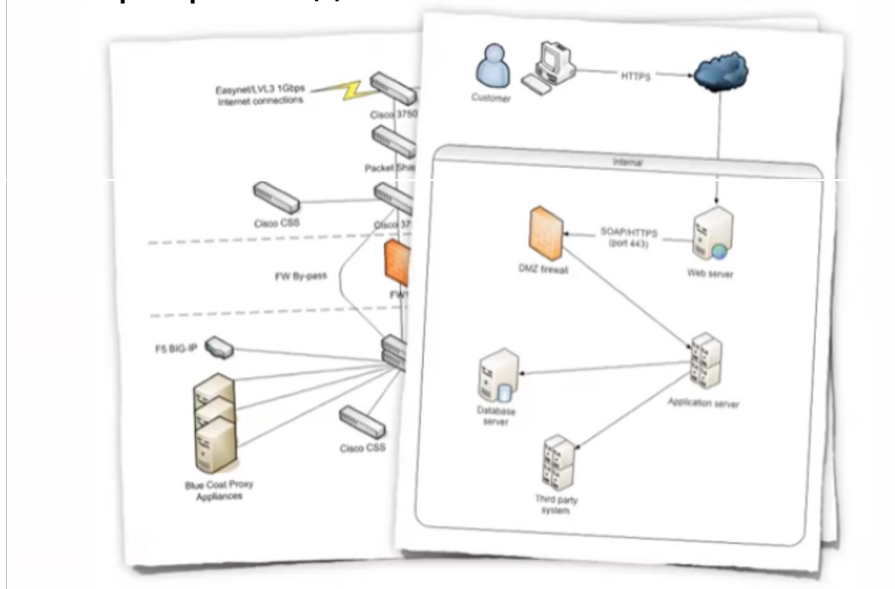
- Детальность и точность для критически важных компонентов
- Многие разработчики и тех. специалисты знакомы с UML
- Только для разработчиков и аналитиков, для бизнес-заказчиков не подойдет

UML фокус

- Объектно-ориентированные концепции
- Детали реализации
- Проектирование программных систем: классы, методы, интерфейсы

Инфраструктурные диаграммы

Диаграммы, которые описывают инфраструктуру
Топология сети, взаимодействие физических машин, в каких стойках
размещаются сервера и т.д.



Вопросы?

 если есть вопросы

 если вопросов нет

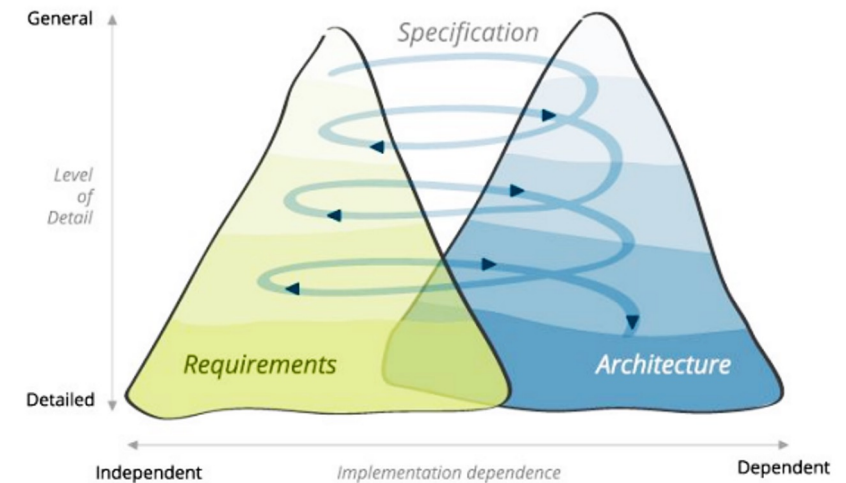
Документирование архитектурных решений

Зачем нужен архитектор?

Принятие архитектурных решений:

Закон 1. «В архитектуре всё — **компромисс**»

Закон 2. «**Почему** в архитектуре важнее, чем **как**»

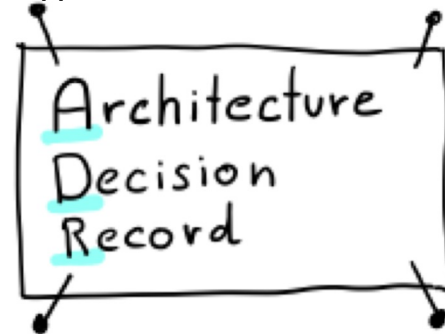


ADR

Architecture decision record (ADR) / Architecture decision log (ADL) — это регулярная фиксация принятых и непринятых в ходе разработки программного обеспечения решений, затрагивающих дизайн, проектирование, выбор инструментов и подходов, и отвечающих определенным функциональным или нефункциональным требованиям. Вместе с решением фиксируется его контекст, последствия и альтернативы, из которых делался выбор.

Это позволяет:

- Повысить качество решения
- Ответить на вопрос, **почему** было принято такое решение



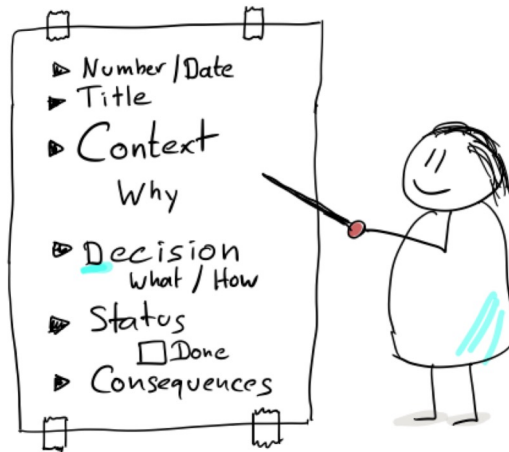
<https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>



ADR

ADR обычно включает в себя:

- Контекст проблемы, которую решаем
- Возможные альтернативы решения
- Решение
- Последствия решения
- Риски, сильные стороны, компромиссы и т.д.



 <https://github.com/joelparkerhenderson/architecture-decision-record>

Пример ADR

Decision record template by Michael Nygard

This is the template in [Documenting architecture decisions - Michael Nygard](#). You can use [adr-tools](#) for managing the ADR files.

In each ADR file, write these sections:

Title

Status

What is the status, such as proposed, accepted, rejected, deprecated, superseded, etc.?

Context

What is the issue that we're seeing that is motivating this decision or change?

Decision

What is the change that we're proposing and/or doing?

Consequences

What becomes easier or more difficult to do because of this change?

Файл ADR обычно именуется с указанием id решения и краткого описания:

NNN-краткое-описание.md

Пример:

001-выбор-базы-данных.md

002-использование-REST-API.md



<https://github.com/joelparkerhenderson/architecture-decision-record/tree/main/locales/en/templates/decision-record-template-by-michael-nygard#decision-record-template-by-michael-nygard>



Ведение лога решений

DECISION LOG					
ID	CATEGORY	DECISION	RESPONSIBLE PARTY	DECISION DATE	COMMENTS
D001	Approval	CRQ015-WP1 approval agreed at the 15 October Joint Project Board. Follow-on action on S. Dole to arrange signatures, and complete project documentation updates.	Joint Project Board	15-Oct	Change is required to fix defect SD0041.
D002	Process	Decision made by S. Dole and F. Smith on daily stand up to complete 'nice to have' defects after go live.	S. Dole and F. Smith	10-Nov	See defect log in sharepoint.
D003	Technical	Dev team have confirmed decision to use jquery plugin for location finder.	A. Peer	10-Nov	Alternatives considered are documented in the Tech Design.
D004	Technical	Sec Ops have reviewed pen test results and agreed to go ahead with implementation of JAX SaaS tool. Vulnerabilities are minor and will be removed with upgrade scheduled for Jan.	D. Orey	13-Oct	See notes of sec ops meeting 10 October.

Почему нужна документация архитектуры

- Архитектура только «в головах».

Bus factor = 1 - критичные знания только у одного человека, передача знаний, отсутствие единого понимания системы у команды. Дублирование решений, накопление технического долга.

- Микросервисная архитектура и ее масштаб

Netflix: 700+ микросервисов

Uber: 4000+ микросервисов

Amazon: десятки тысяч сервисов

Без документации невозможно понять потоки данных и зависимости.

Распределенные транзакции, eventual consistency, синхронные/асинхронные паттерны коммуникации, service mesh.

- Распределенные команды и DevOps

Разные часовые пояса и культуры, удаленная работа, десятки deploy в день.

Нужна документация, которая успевает за изменениями

Живая документация. Требования

Актуальная документация

- Обновляется вместе с кодом
- Версионруется в Git вместе с проектом
- Проверяется в CI/CD pipeline
- Автоматически публикуется

Примеры инструментов:

- Swagger/OpenAPI для API
- Spring REST Docs из тестов
- PlantUML в markdown файлах
- Architecture Decision Records (ADR)

Живая документация. Требования

Многоуровневая

- Разные представления архитектуры для разных ролей
- Связи и навигация между различными слоями архитектуры
- Возможность приближения/отдаления (zoom-in/zoom-out)

Формальная, но понятная

- Известная нотация
- Понятна всем участникам – единый язык описания архитектуры

Вопросы?

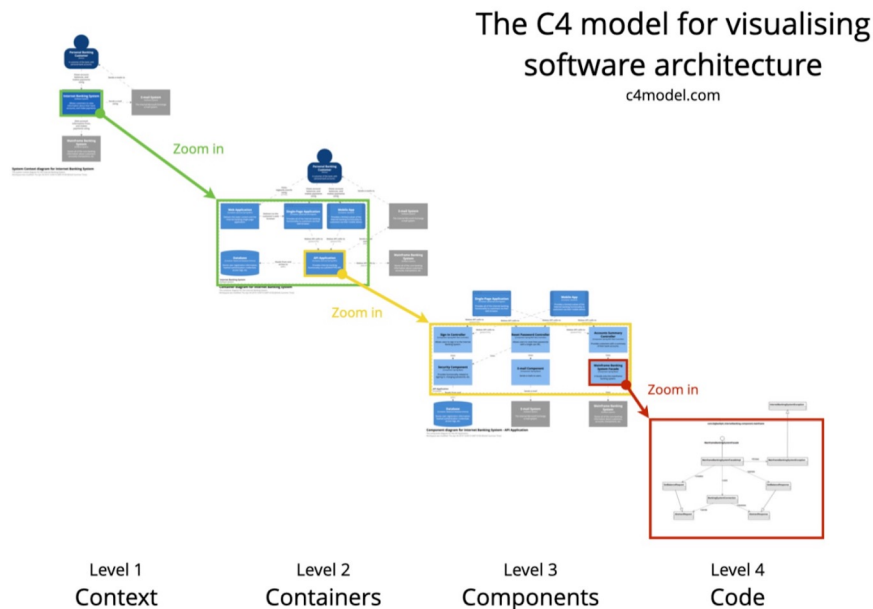
 если есть вопросы

 если вопросов нет

C4 модель представлений

C4 модель Саймона Брауна

Саймон Браун в 2015 году представил модель C4 — идейную наследницу 4+1, но чуть более простую и современную версию. Больше подходящую для software architecture.



Нотация vs Абстракция

Любая диаграмма представляет собой набор абстракций, которые между собой каким-то образом связаны, описанных на каком-то языке (нотация).

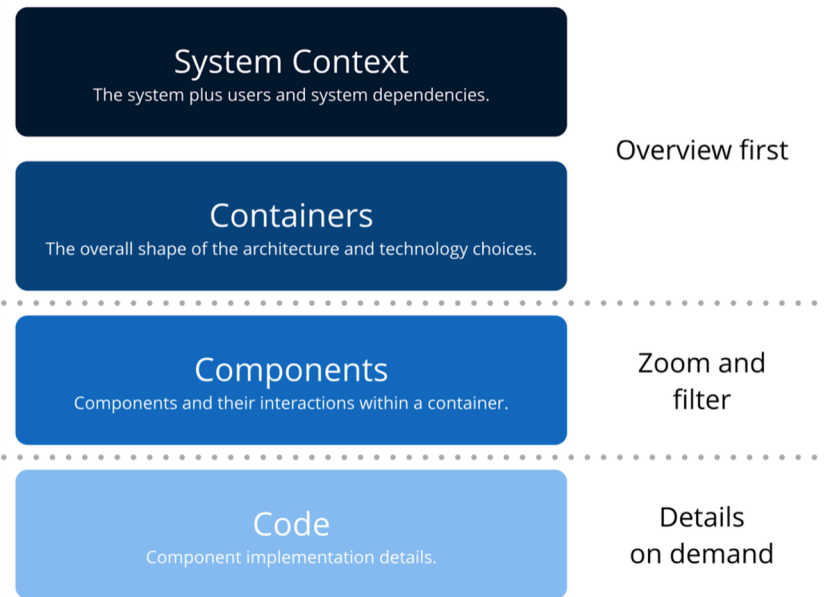
Идея Саймона заключалась в том, что не столь важна нотация, а важно про какие абстракции рассказывается на диаграмме.

В конце концов квадратики и стрелочки можно подписать и добавить легенду.

C4 модель Саймона Брауна

В рамках парадигмы C4 предлагается 4 уровня представлений в архитектуре:

- Контекст
- Контейнер
- Компонент
- Код



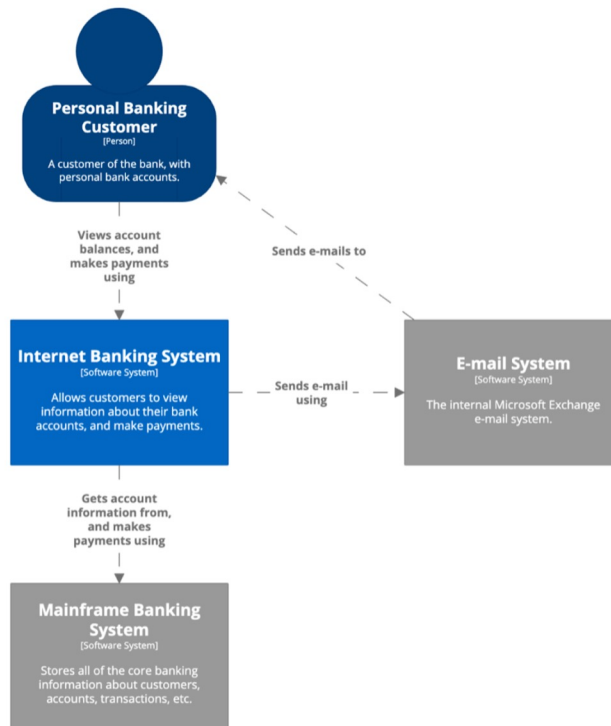
Контекст

Контекстная диаграмма – это диаграмма, которая предоставляет первоначальное приближение для системы, показывает как система связана с окружающим миром.

Это диаграмма, которая должна отвечать на вопросы:

- Про что наше приложение?
- Кто использует его?
- Как приложение интегрировано в окружающую среду?

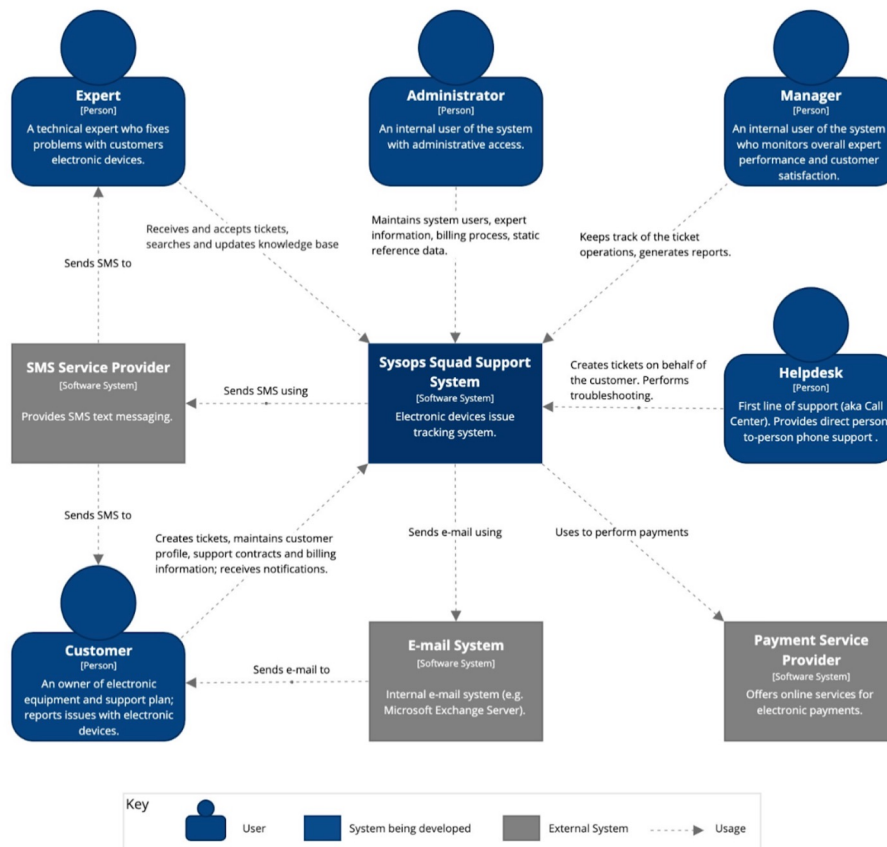
Контекст



System Context diagram for Internet Banking System

The system context diagram for the Internet Banking System.
Workspace last modified: Wed Feb 05 2020 09:33:36 GMT+0100 (Central European Standard Time)

Контекст



Контейнер

Контейнер – это что-то, что «запускает» приложение или хранит данные.

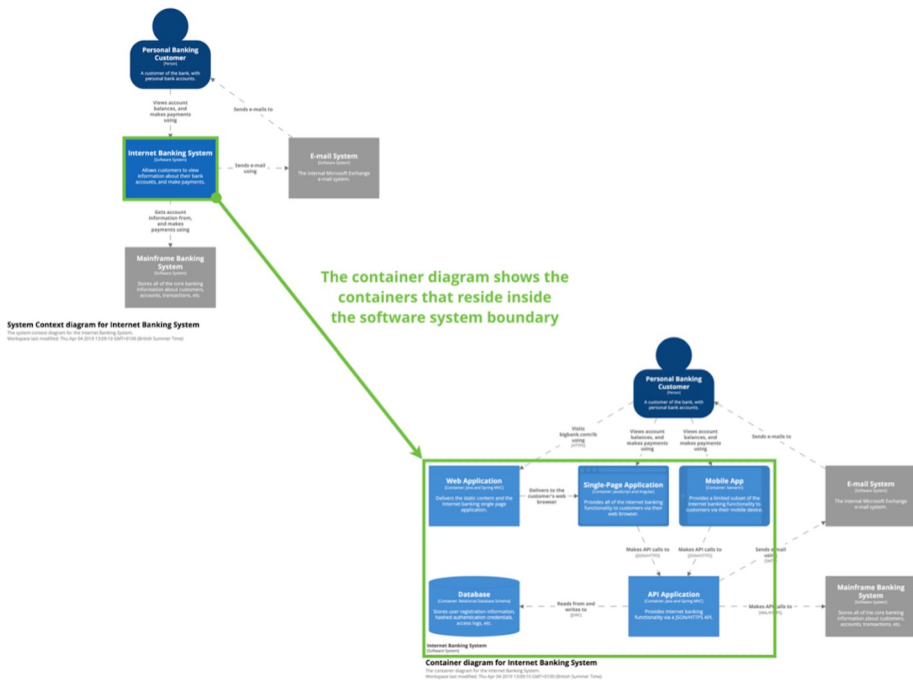
Примеры контейнеров:

- Серверное веб-приложение
- Клиентское веб-приложение
- Мобильное приложение
- Микросервис
- База данных
- Файловая система
- Шелл скрипт
- И т.д.

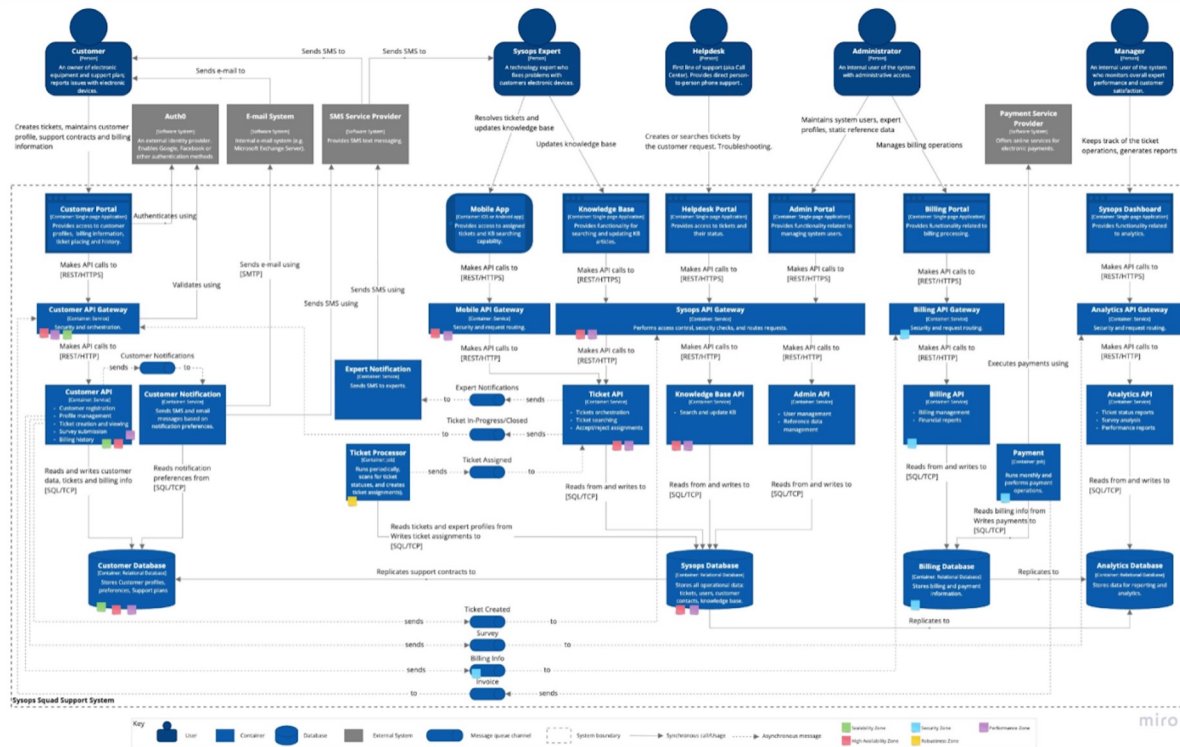
Основной критерий: **контейнер** – это что-то, что можно задеплоить независимо друг от друга.

Контейнер

Диаграмма контейнеров позволяет посмотреть, из чего состоит система.



Контейнер



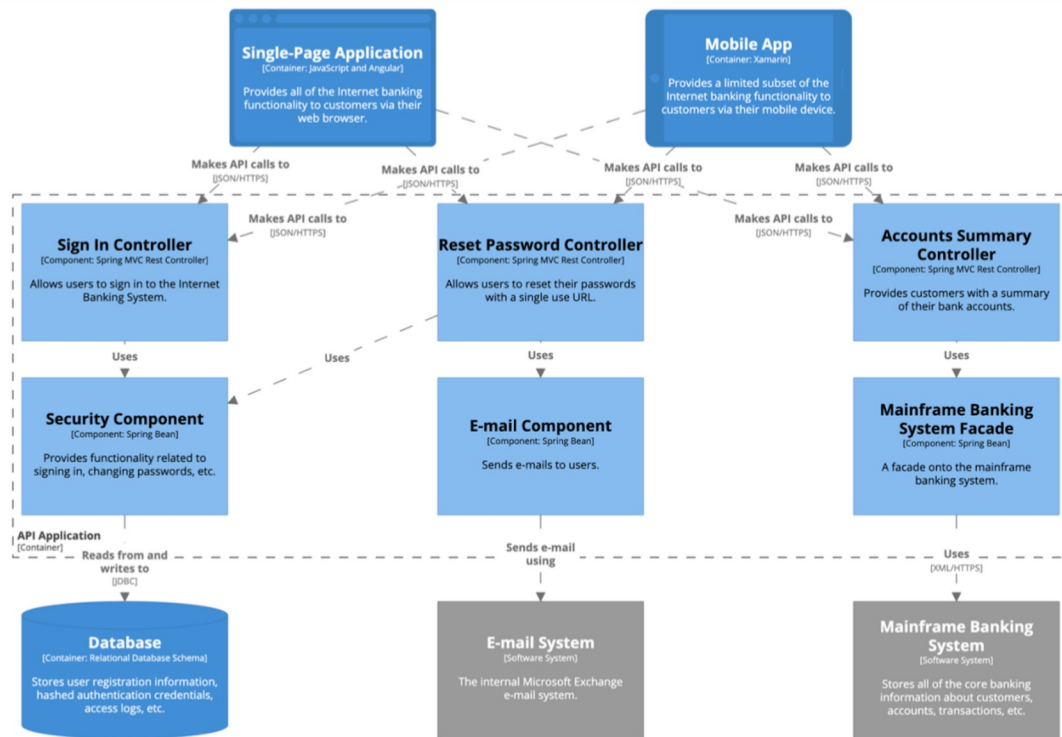
Компонент

Компонент – это некая группа функциональности, которые объединены и спрятаны (инкапсулированы) за строго определенный интерфейс.

Например:

- В C++, Java компонентом будет некоторое множество классов и интерфейсов
- В JS компонентом может быть модуль
- В Haskell компонентом тоже может быть группа типов и функций, объединенных в модуль и т.д.

Диаграмма КОМПОНЕНТОВ



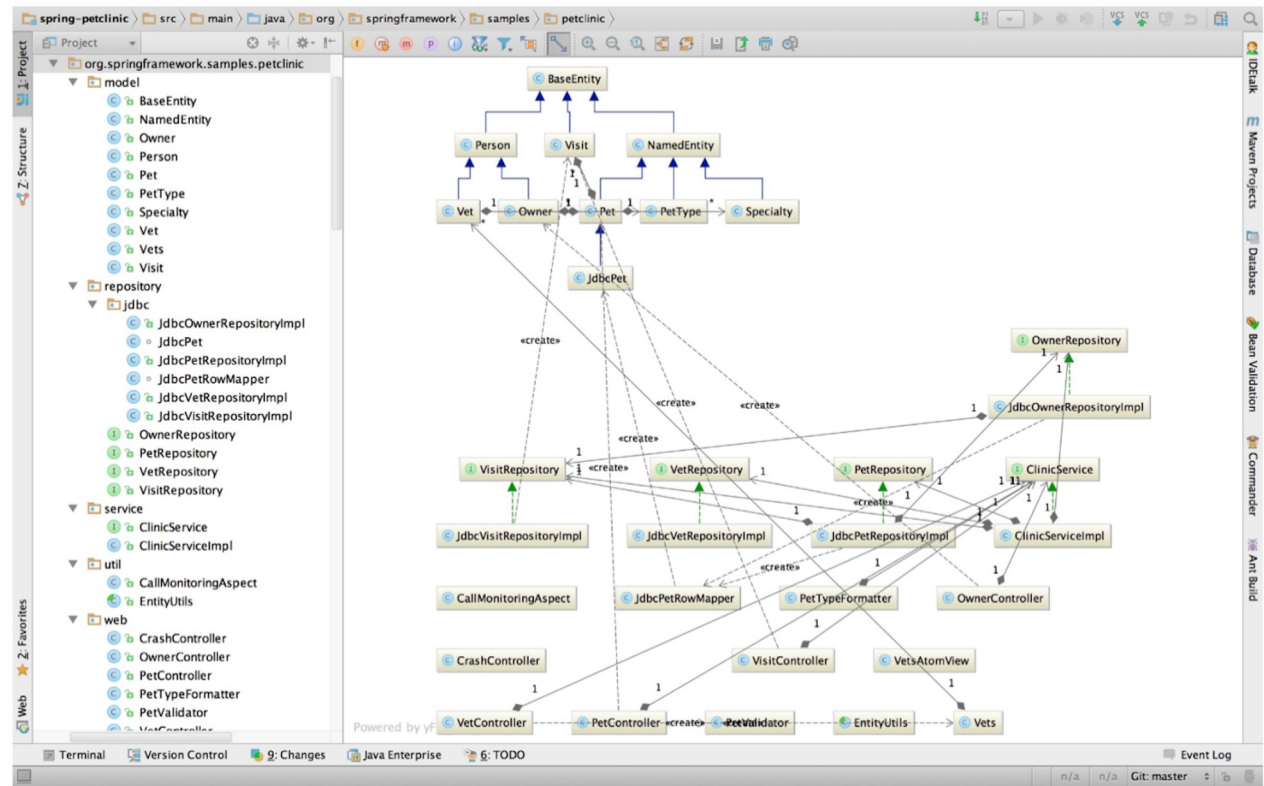
Component diagram for Internet Banking System - API Application

The component diagram for the API Application.
Workspace last modified: Thu Apr 04 2019 13:09:10 GMT+0100 (British Summer Time)

Код

Код – это объекты уровня языка программирования или фреймворка: классы, методы, функции и т.д.

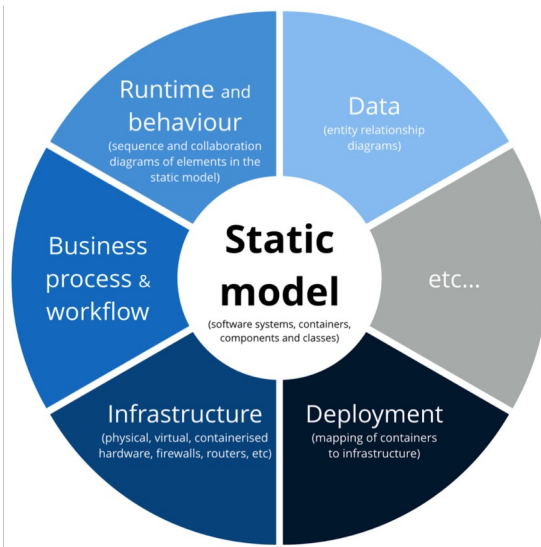
Диаграмма уровня кода



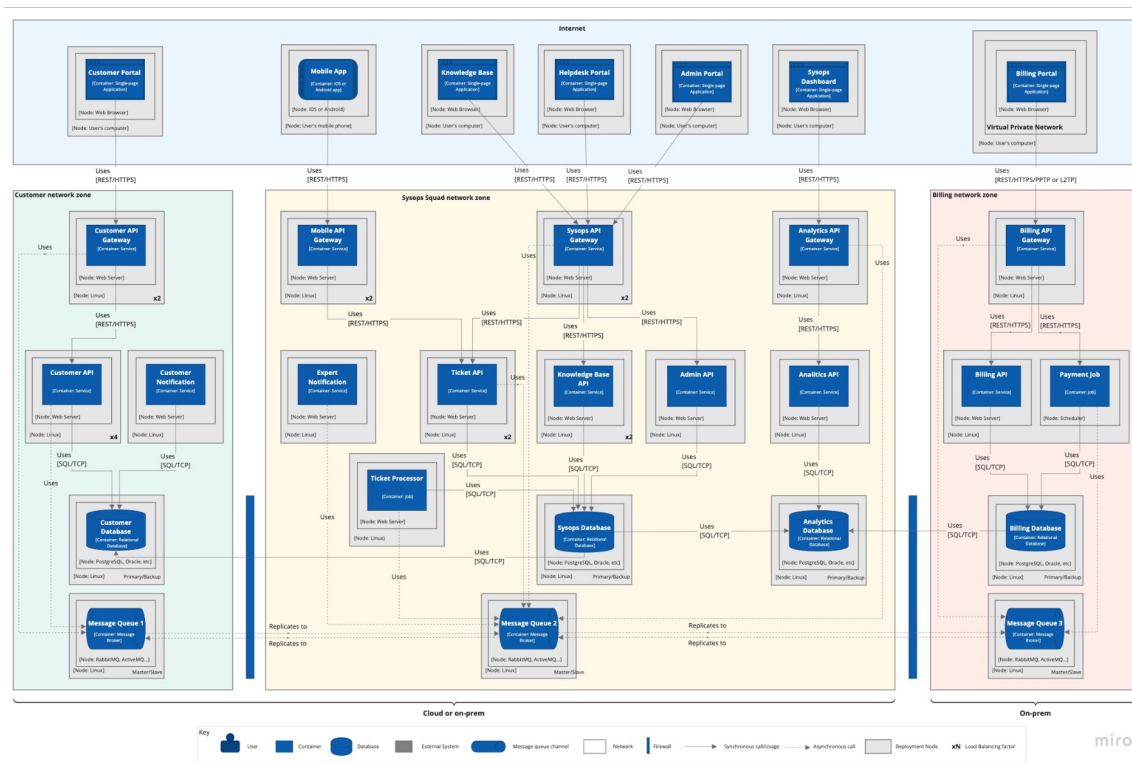
Другие диаграммы

Диаграмма контекста, контейнеров, компонентов и кода позволяет получить статичную картинку того, как выглядит архитектура сейчас.

Но помимо этих диаграмм, нас могут и будут интересовать другие аспекты архитектуры.



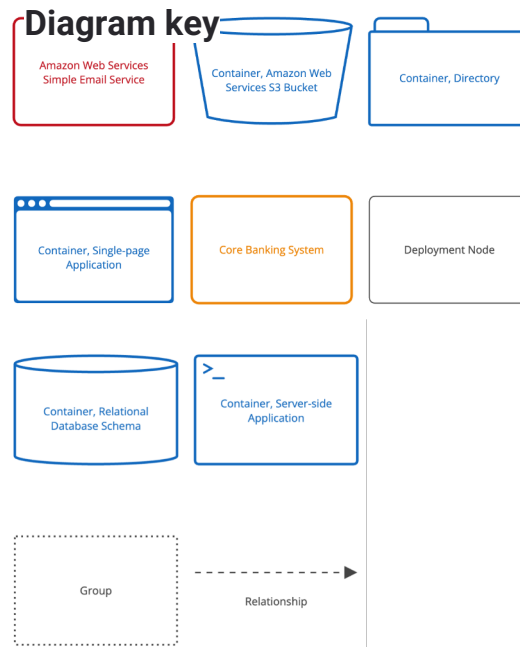
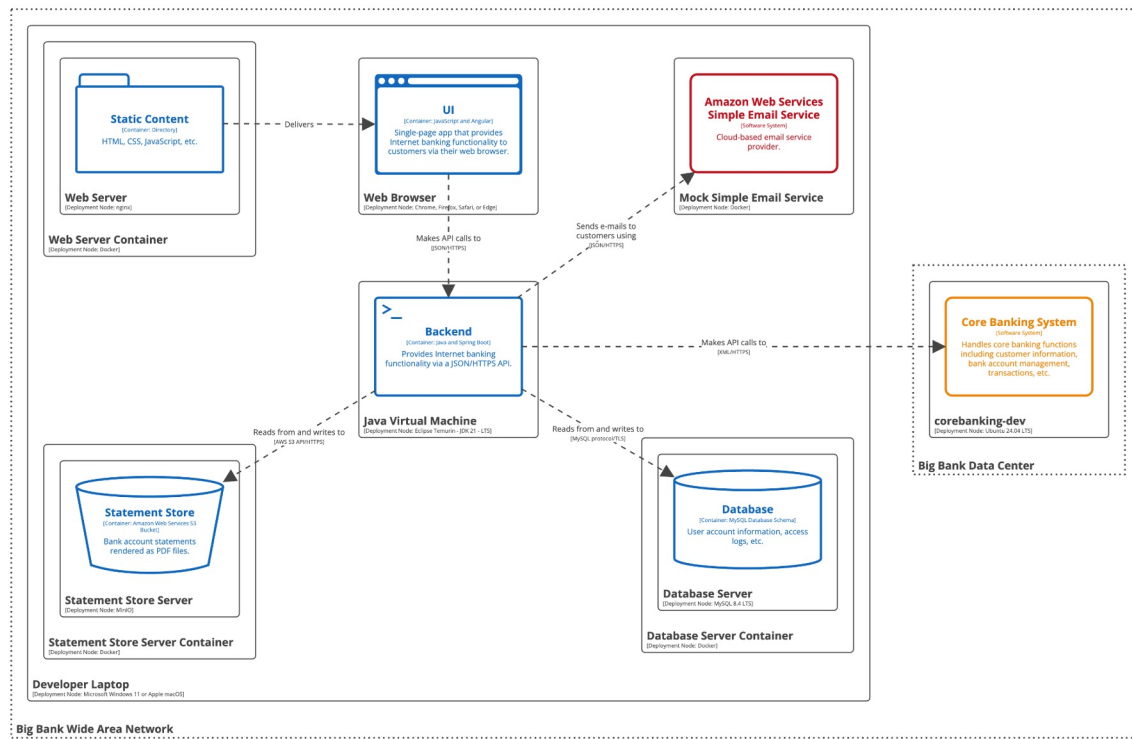
Деплоймент диаграмма



<https://github.com/team7katas/sysopsquad/blob/main/images/deployment.jpg>



Деплоймент диаграмма



Deployment View: Internet Banking System - Development

An example development deployment scenario for the Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0



<https://c4model.com/diagrams/deployment>



Вопросы?

 если есть вопросы

 если вопросов нет

Инструменты

1. Инструменты для моделирования

- PlantUML
- Mermaid
- d2 diagrams
- Structurizr
- drawio
- yEd
- Archi

2. Технические детали

- бесплатны, Structurizr - freemium mode
- самое сложное и универсальное - Archi
- тех. архитектура – UML, C4

Инструменты

- <https://lucid.app/> - Lucid Chart
- <https://app.diagrams.net/> - Draw.io
- <https://www.omnigroup.com/omnigraffle> - Omnigraffle
- <https://plantuml.com/> - PlantUML
- <https://mermaid-js.github.io/> - Mermaid
- <https://www.archimatetool.com/> - инструмент для работы с Archimate
- <https://miro.com/> - Miro
- <https://www.yworks.com/products/yed> - YeD
- <https://www.visual-paradigm.com/>
- <https://structurizr.com/>

Best practice

- **Актуальность**
- **Фокус на аудитории при формировании диаграммы**
- **Подходящий уровень детализации**

**Подведём
итоги занятия**

Слайд с тезисами

Подведем итоги

1. Атрибуты качества архитектуры: характеристики, сценарии. Нет универсальных для всех систем, всегда измеримы(абстрактной производительности не бывает!)
2. Документация по архитектуре решения должна быть актуальной, используемой, у нее есть целевая аудитория.
ADR/ADL, различные форматы
3. Архитектурные представления описывают разные аспекты архитектуры
4. C4 model – современный и удобный способ описания архитектуры



В C4 model можно
визуализировать
функциональную архитектуру
информационной системы?



В C4 model можно
визуализировать
функциональную архитектуру
информационной системы?

- Нет, C4 model предназначена
для технических представлений

Цели вебинара

Отправьте в чат номера достигнутых целей

1. Поймете, что такое атрибуты качества архитектуры и как их формулировать
2. Узнаете как документировать архитектурные решения ADR/ADL в работе
3. Узнаете представления архитектуры и сможете их применять

Рефлексия



С какими впечатлениями уходите с вебинара?



**Как будете применять на практике то,
что узнали на вебинаре?**

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

Приглашаем на следующий вебинар: “От монолита к микросервису”



Ссылка на вебинар будет
в ЛК за 15 минут



Материалы
к занятию в ЛК —
можно изучать



Обязательный материал
обозначен красной лентой